# Design and Analysis of a Simplified Pipelined MIPS Processor with Forwarding Logic

Yu-Kuan Lin

#017515004

MSEE

2025/4/2

## I. Introduction

This report presents the design and implementation of a Verilog-based simulation model that emulates the behavior of a simplified 32-bit pipelined MIPS processor, specifically optimized for executing a subset of the MIPS instruction set architecture (ISA) intended for performing dot product computations. The project includes two distinct pipeline versions: one without forwarding and another enhanced with forwarding mechanisms. The objective is to demonstrate how pipeline hazards—particularly data hazards—affect performance and how forwarding logic can significantly mitigate these hazards. While not a true hardware implementation of a complete MIPS CPU, this simulation captures essential pipeline behavior and provides valuable insights into pipeline optimization techniques. The modifications from the given MIPS intructions subset, such as substituting "addi $7, $7, -1" with "subi $7, $7, 1" and adjusting instruction sequences, were deliberately made to simplify handling unsigned arithmetic and minimize data hazards.

## II. Description of the Design

### *A. 5-Stage Pipelined MIPS without Forwarding Logic*

Initially, a pipeline without forwarding logic was implemented to illustrate basic pipeline behavior and the negative impacts of pipeline hazards. This simplified pipeline follows a classic five-stage structure: Instruction Fetch (IF), Instruction Decode (ID), Execution (EX), Memory Access (MEM), and Write Back (WB). In this version, RAW hazards introduce stalls that significantly degrade performance.

It is important to clarify that this design does not represent a genuine MIPS CPU hardware implementation; rather, it is a Verilog simulation emulating MIPS pipeline behavior. Due to the simulation nature of this design, which does not include actual hardware components such as adders or an ALU, it is not feasible to create a corresponding block diagram. Also, unlike actual MIPS processors, this design completes the determination of control flow instructions (branch, jump, jump register) and calculates corresponding addresses entirely within the IF stage using lengthy combinational logic:

```
assign takebranch_04 = ((IFIDIR_04[31:26]==BEQ_04) && (Regs_04[IFIDIR_04[25:21]]==Regs_04[IFIDIR_04[20:16]]));
assign takejump_04 = (IFIDIR_04[31:26]==J_04);
assign takejr_04   = ((IFIDIR_04[5:0]==JR_04)&&(IFIDIR_04[31:26]==ALUop_04));
```

This approach significantly differs from an actual MIPS processor, where these tasks are typically divided across IF and ID stages, or even further extended into the EX stage. Using extensive combinational logic at the IF stage is impractical for real hardware, as it creates timing constraints that could lengthen pipeline cycles and limit maximum operating frequency.

Additionally, control flow instructions (beq, j, jr) cause a complete flush in the subsequent pipeline cycle, inserting a single-cycle stall to ensure correct execution:

```
if (takejump_04) begin
     IFIDIR_04 <= no_op_04;  \\no_op_04=32'd0
      PC_04    <= ({6'b000000, IFIDIR_04[25:0]}<<2); end
else if (takejr_04) begin
     IFIDIR_04 <= no_op_04;
     PC_04     <= Regs_04[31]; end
  else if (takebranch_04) begin
     IFIDIR_04 <= no_op_04;
     PC_04     <= PC_04 + 4 + ({{16{IFIDIR_04[15]}},IFIDIR_04[15:0]}<<2);  end
```

The implemented instruction subset includes arithmetic operations (add, mul), immediate arithmetic operations (addi, subi), memory operations (lw, sw), and control flow instructions (beq, j, jr). The "subi"

instruction replaces an original assignment requirement of "addi $7, $7, -1" to simplify signed arithmetic handling. Adjustments to instruction sequences were made strategically to prevent data hazards, particularly within loop control logic, thus simplifying hazard management and improving pipeline efficiency.

### *B. 5-Stage Pipelined MIPS with Forwarding Logic*

In the second pipeline version, forwarding logic was added to address RAW hazards effectively. This version introduced additional combinational logic to dynamically detect data hazards and forward operand values directly from pipeline registers (EX/MEM or MEM/WB) to the EX stage, thereby bypassing unnecessary stalls. Specific forwarding logic includes:

- bypassAfromMEM_04 and bypassBfromMEM_04: Detect RAW hazards when an instruction in EX depends on an instruction in MEM:
  **assign bypassAfromMEM_04 = (IDEXrs_04 == EXMEMrd_04) & (IDEXrs_04 != 0) & (EXMEMop_04==ALUop_04);**
  **assign bypassBfromMEM_04 = (IDEXrt_04 == EXMEMrd_04) & (IDEXrt_04 != 0) & (EXMEMop_04==ALUop_04);**
- bypassAfromALUinWB_04 and bypassBfromALUinWB_04: Handle RAW hazards from arithmetic instructions in WB:
  **assign bypassAfromALUinWB_04=(IDEXrs_04==MEMWBrd_04)&(IDEXrs_04 !=0)&(MEMWBop_04==ALUop_04);**
  **assign bypassBfromALUinWB_04=(IDEXrt_04==MEMWBrd_04)&(IDEXrt_04 != 0)&(MEMWBop_04==ALUop_04);**
- bypassAfromLWinWB_04 and bypassBfromLWinWB_04: Address RAW hazards from load instructions:
  **assign bypassAfromLWinWB_04 = (IDEXrs_04 == MEMWBIR_04[20:16]) & (IDEXrs_04 != 0) & (MEMWBop_04==LW_04);**
  **assign bypassBfromLWinWB_04 = (IDEXrt_04 == MEMWBIR_04[20:16]) & (IDEXrt_04 != 0) & (MEMWBop_04==LW_04);**

The stall logic explicitly inserts bubbles at the EX stage to handle load-use hazards not resolvable by forwarding alone. This bubble insertion is required because a RAW hazard resulting from a load instruction cannot be fully resolved with forwarding alone, as the loaded data from memory is only available after the MEM stage. Thus, the instruction following the load must be stalled for one cycle to wait for the data to become available:

**assign stall_04=((IDEXop_04==LW_04)&&((IDEXIR_04[20:16]==IFIDIR_04[25:21])||(IDEXIR_04[20:16]==IFIDIR_04[20:16])))**

Both pipeline designs were simulated in Verilog and validated through detailed waveform analysis, confirming accurate pipeline behavior and effective hazard resolution. The forwarding mechanism notably improved performance by minimizing stalls, highlighting the critical role of hazard management techniques in pipeline processor design.

## III. Simulation Results

Two testbenches were designed to validate the pipelined processor's behavior under different scenarios. The instruction subset for the pipeline with forwarding differs from the assignment requirements to simplify unsigned arithmetic handling and reduce complexity:

- Assignment-required sequence: add, beq, lw, lw, mul, add, addi, addi, addi, j, jr
- Actual implemented sequence (forwarding version):add, beq, lw, lw, mul, add, subi, addi, addi, j, jr

The "addi $7, $7, -1" instruction required by the assignment was replaced with "subi $7, $7, 1" to simplify signed arithmetic handling. Additionally, to prevent the value of register r7 from being prematurely read by the subsequent beq instruction after looping back, the subi instruction was moved two steps earlier in the sequence.

For the pipeline without forwarding, manual insertion of no-operation instructions (no_op) was necessary within the testbench to prevent data hazards. This ensured the pipeline could function correctly despite lacking forwarding logic. The forwarding-enabled testbench did not require these manual insertions, as hazards were resolved through the pipeline's built-in forwarding logic.

The vectors used for the dot product calculation are:

- Vector A: [0, 1, 7, 5, 1, 5, 0, 0, 4]
- Vector B: [4, 5, 1, 9, 5, 9, 4, 4, 8]

The correct dot product result calculated from these vectors is **139**.

Fig. 1 and Fig. 2 show the waveform simulation results for the no forwarding and with forwarding designs, respectively. The key signals from top to bottom are:

1. **R1_04** to **R7_04** – The values of registers 1 through 7
2. **IFIDIR_04**, **IDEXA_04**, **EXMEMIR_04**, **MEMWBIR_04** – Instructions in each pipeline stage
3. **PC_04** – The current program counter
4. **stall_04** – A flag signal indicating whether a bubble is inserted (present only in the with forwarding design)
5. **bypassBfromLWinWB_04** – A flag signal indicating whether a WB-stage lw value is being forwarded to EX (only in the with forwarding design)
6. **takejump_04**, **takebranch_04**, **takejr_04** – Flag signals for control flow instructions j, beq, and jr

From the value of R1_04 at the far right of both Fig. 1 and Fig. 2, it is evident that both simulations successfully computed the correct dot product result, indicating both designs are functionally correct. In Fig. 1, the signals **takejump_04**, **takebranch_04**, **takejr_04** demonstrate that control flow instruction flags are correctly triggered, and the pipeline performs the corresponding control flow operations. At the bottom-right of both Fig. 1 and Fig. 2, **takebranch_04** and **takejr_04** can be observed toggling alternately. This loop is caused by register r31 (the return address) being set to 0, causing jr to redirect control flow to the initial add instruction. The next instruction is beq, which branches again because r7 equals 0, thereby forming a loop that triggers **takebranch_04** and **takejr_04** repeatedly. In Fig. 2, the signals **stall_04** and **bypassBfromLWinWB_04** demonstrate that the forwarding logic is correctly activated, successfully passing the lw result from the WB stage to the EX stage for use in the mul operation.

Fig. 3 shows a zoomed-in portion of the no forwarding waveform during one of the loops. It clearly demonstrates that the manually inserted no_op instructions in the testbench are correctly fetched and executed, effectively avoiding data hazards. By observing the transitions of **IFIDIR_04**, **IDEXA_04**, **EXMEMIR_04**, and **MEMWBIR_04**, it is evident that the five-stage pipeline behaves as expected, with each instruction progressing correctly through the pipeline stages.

Fig. 4 presents the corresponding section of the waveform of simulation with forwarding, focusing on the sequence involving lw, lw, mul, and add instructions. In this figure, the assertion of the **stall_04** signal causes a bubble to be inserted into the EX stage, as shown by the presence of a no_op instruction. Consequently, the mul instruction remains stalled in the ID stage for one cycle before advancing. This behavior confirms the correct functioning of the stall insertion logic in response to load-use hazards.

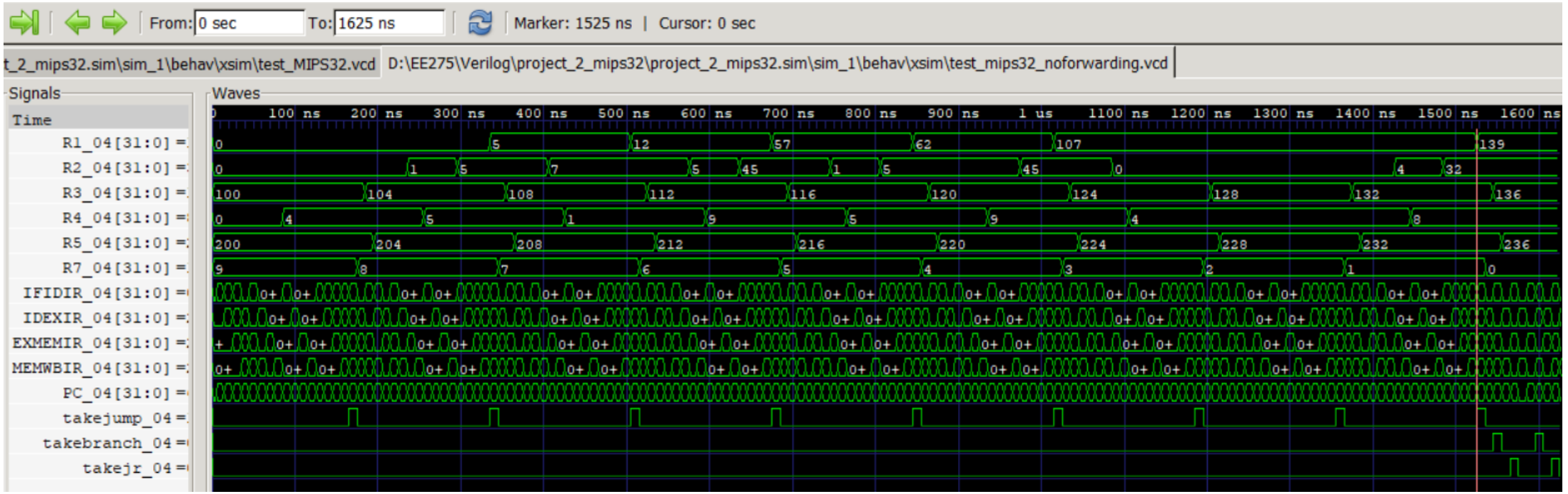


Fig. 1. MIPS without Forwarding simulation waveform

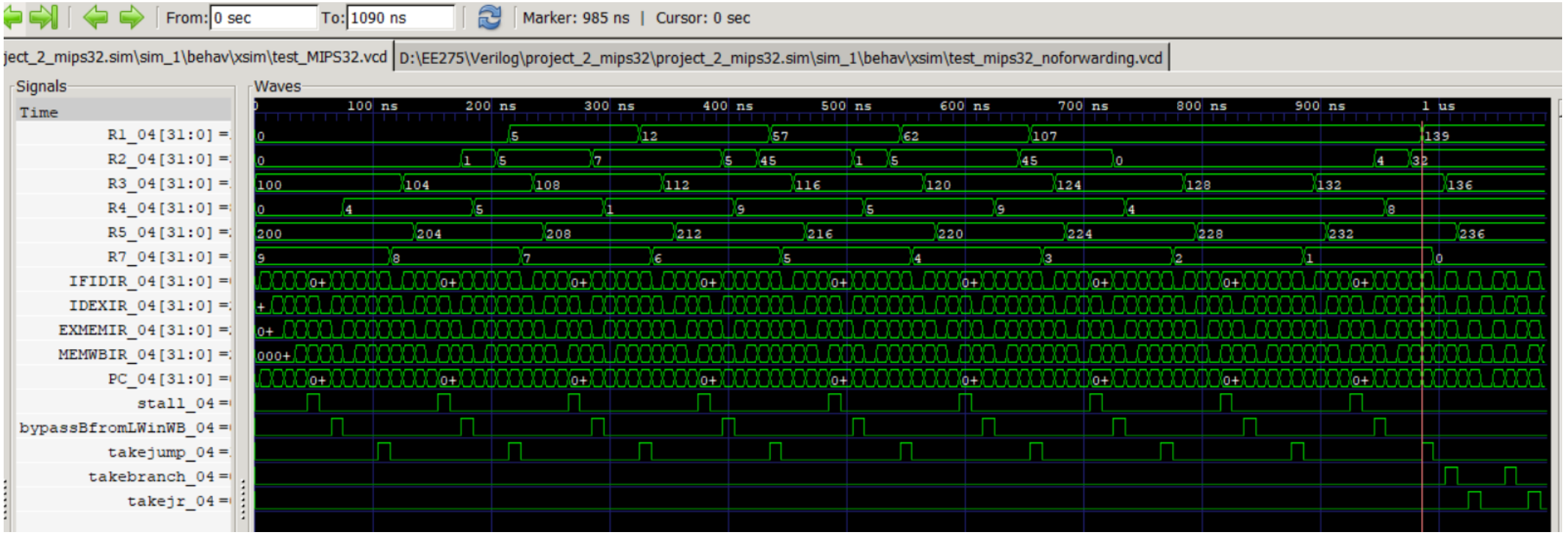


Fig. 2. MIPS with Forwarding simulation waveform

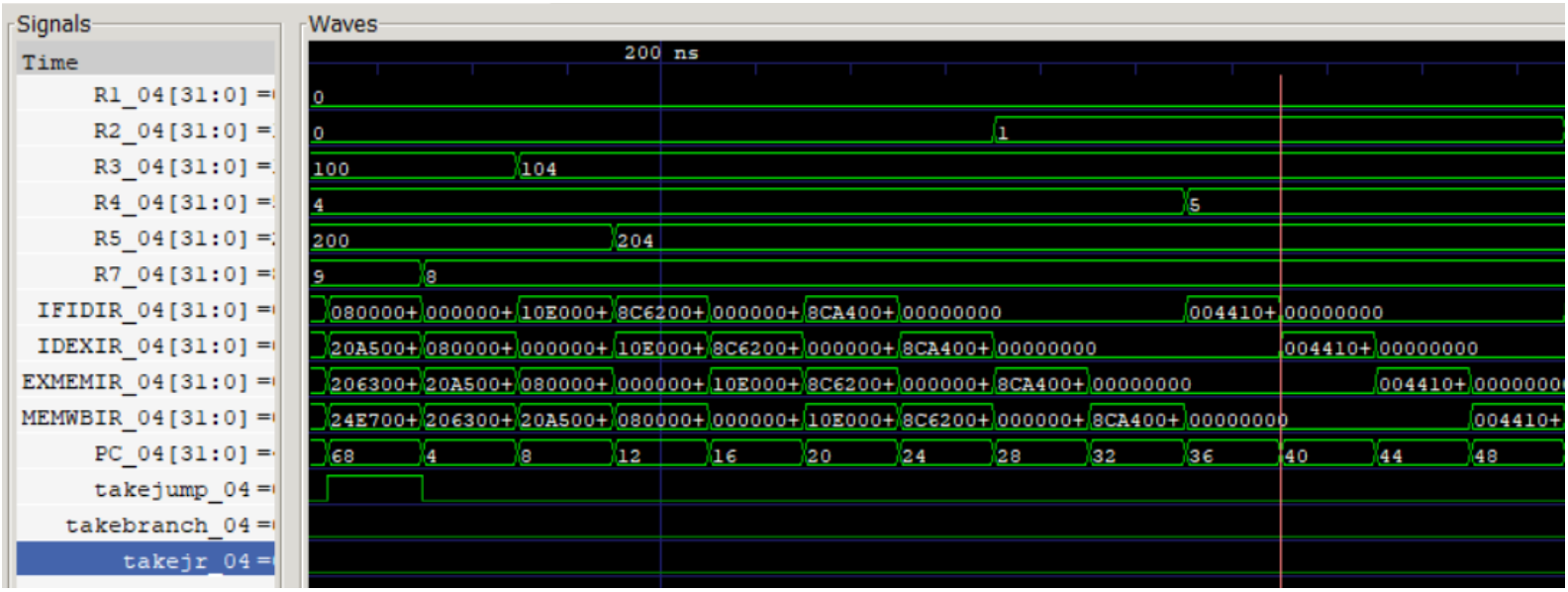


Fig. 3. MIPS without Forwarding simulation waveform (partial)

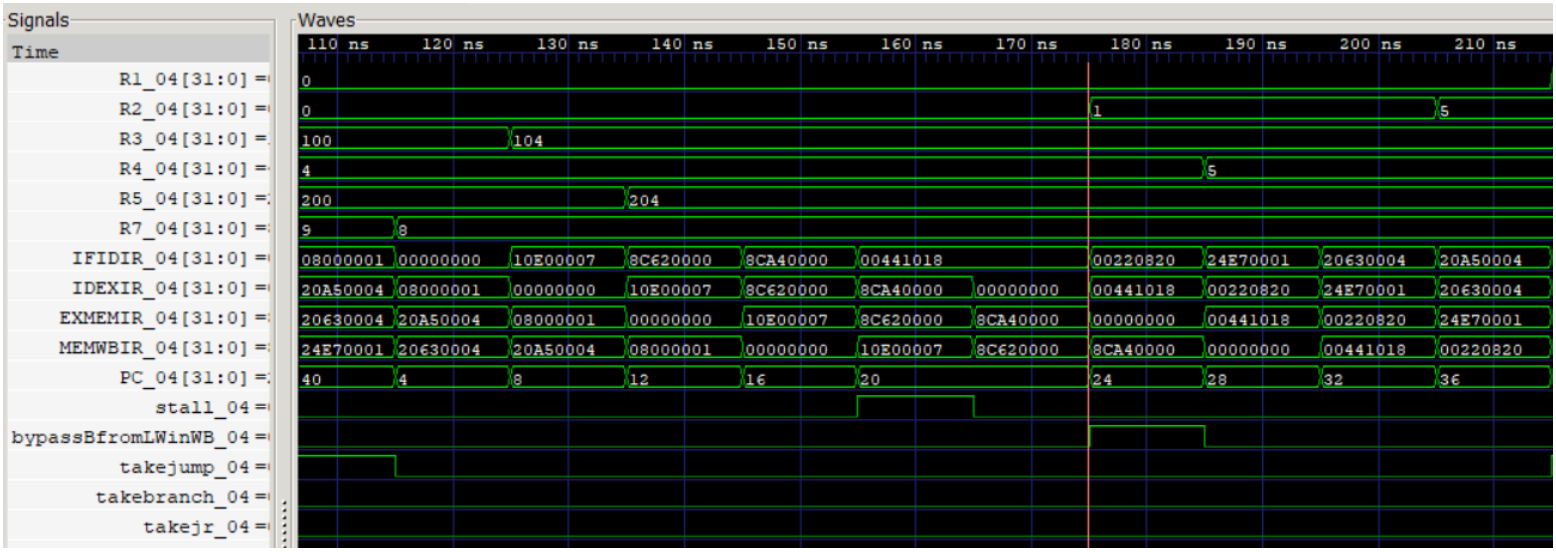


Fig. 4. MIPS with Forwarding simulation waveform (partial)

## IV. Discussion and Conclusion

On the far right side of Fig. 1 and Fig. 2, a red marker indicates the moment when the dot product calculation completes—that is, when the value of R1 is updated to 139. At the top of Fig. 1 and Fig. 2, the timestamp at the red marker highlights the completion time for each design: the MIPS without forwarding design finishes the calculation at 1525 ns, while the MIPS with forwarding design completes it at 985 ns. Given that each pipeline cycle spans 10 ns, this means the forwarding version finishes **54 cycles** earlier than the version without forwarding. This demonstrates a substantial performance gain. Using the inverse ratio of their execution times, we compute the speedup of the forwarding-enabled pipeline as:

$$\text{Speedup} = \frac{Time_{no\ forwarding}}{Time_{forwarding}} = \frac{1525}{985} \approx 1.548$$

Although the design successfully implements data forwarding and demonstrates significant performance improvement, it does not include the 2-bit branch predictor component originally required in the assignment. This omission was a deliberate decision made due to time constraints and complexity, in order to focus development efforts on validating forwarding logic. Nevertheless, the results confirm that even partial hazard resolution techniques such as forwarding can have a measurable impact on pipeline performance and provide a strong foundation for further enhancement

## Appendix A: 5-Stage Pipelined MIPS without Forwarding Design Code

```
`timescale 1ns / 1ps
module pipe_MIPS32_noforward(clock);
input clock;

 parameter LW_04    = 6'b100011, // 35
           SW_04    = 6'b101011, // 43
           BEQ_04   = 6'b000100, //  4
           ADDI_04  = 6'b001000, // 8
           SUBI_04  = 6'b001001, // 9
           J_04     = 6'b000010, //  2
           ALUop_04 = 6'b000000, // R-type
           no_op_04 = 32'b000000_00000_00000_00000_00000_000000;
 parameter
           MUL_04   = 6'b011000, // 24
           ADD_04   = 6'b100000, // 32
           SUB_04   = 6'b100010, // 34
           JR_04    = 6'b001000; // 8

 reg [31:0] PC_04;
 reg [31:0] Regs_04[0:31], IMemory_04[0:1023], DMemory_04[0:1023];
 reg [31:0] IFIDIR_04;
 reg [31:0] IDEXA_04, IDEXB_04, IDEXIR_04;
 reg [31:0] EXMEMIR_04, EXMEMALUOut_04, EXMEMB_04;
 reg [31:0] MEMWBIR_04, MEMWBValue_04;

 wire [4:0] IDEXrs_04, IDEXrt_04, EXMEMrd_04, MEMWBrd_04;
 wire [5:0] IDEXop_04, EXMEMop_04, MEMWBop_04;

 wire takebranch_04, takejump_04, takejr_04;
 wire [31:0] Ain_04, Bin_04;

 assign IDEXrs_04 = IDEXIR_04[25:21];
 assign IDEXrt_04 = IDEXIR_04[20:16];
 assign IDEXop_04 = IDEXIR_04[31:26];
 assign EXMEMrd_04 = EXMEMIR_04[15:11];
 assign EXMEMop_04 = EXMEMIR_04[31:26];
 assign MEMWBrd_04 = MEMWBIR_04[15:11];
 assign MEMWBop_04 = MEMWBIR_04[31:26];
 assign takebranch_04 = ((IFIDIR_04[31:26]==BEQ_04) &&
                  (Regs_04[IFIDIR_04[25:21]] == Regs_04[IFIDIR_04[20:16]]));
 assign takejump_04 = (IFIDIR_04[31:26]==J_04);
 assign takejr_04   = ((IFIDIR_04[5:0]==JR_04)&&(IFIDIR_04[31:26]==ALUop_04));
 assign Ain_04 = IDEXA_04;
 assign Bin_04 = IDEXB_04;

 always @(posedge clock) begin

    if (takejump_04) begin
      IFIDIR_04 <= no_op_04;
      PC_04     <= ({6'b000000, IFIDIR_04[25:0]}<<2);
    end
    else if (takejr_04) begin
      IFIDIR_04 <= no_op_04;
      PC_04     <= Regs_04[31];
    end
    else if (takebranch_04) begin
```

```
      IFIDIR_04 <= no_op_04;
      PC_04     <= PC_04 + 4 + ({{16{IFIDIR_04[15]}},IFIDIR_04[15:0]}<<2);
    end
    else begin
      IFIDIR_04 <= IMemory_04[PC_04>>2];
      PC_04     <= PC_04 + 4;
    end

    IDEXA_04 <= Regs_04[IFIDIR_04[25:21]];
    IDEXB_04 <= Regs_04[IFIDIR_04[20:16]];
    IDEXIR_04<= IFIDIR_04;

    if (IDEXop_04==LW_04 || IDEXop_04==SW_04) begin
      EXMEMALUOut_04 <= IDEXA_04 + {{16{IDEXIR_04[15]}}, IDEXIR_04[15:0]};
    end
    else if (IDEXop_04==ADDI_04) begin
      EXMEMALUOut_04 <= IDEXA_04 + {{16{IDEXIR_04[15]}}, IDEXIR_04[15:0]};
    end
    else if (IDEXop_04==SUBI_04) begin
      EXMEMALUOut_04 <= IDEXA_04 - {{16{IDEXIR_04[15]}}, IDEXIR_04[15:0]};
    end
    else if (IDEXop_04==ALUop_04) begin
      case(IDEXIR_04[5:0])
        ADD_04:
          EXMEMALUOut_04 <= Ain_04 + Bin_04;
        SUB_04:
          EXMEMALUOut_04 <= Ain_04 - Bin_04;
        MUL_04:
          EXMEMALUOut_04 <= Ain_04 * Bin_04;
        default: EXMEMALUOut_04 <= 32'hx;
      endcase
    end

    EXMEMIR_04 <= IDEXIR_04;
    EXMEMB_04  <= IDEXB_04;

   if (EXMEMop_04==LW_04) begin
     MEMWBValue_04 <= DMemory_04[EXMEMALUOut_04>>2];
   end else if (EXMEMop_04==SW_04) begin
     DMemory_04[EXMEMALUOut_04>>2] <= EXMEMB_04;
   end else if ((EXMEMop_04==ALUop_04)||(EXMEMop_04==ADDI_04)||(EXMEMop_04==SUBI_04)) begin
     MEMWBValue_04 <= EXMEMALUOut_04;
   end
   MEMWBIR_04 <= EXMEMIR_04;

   if (MEMWBop_04==LW_04) begin
     if (MEMWBIR_04[20:16]!=0) Regs_04[MEMWBIR_04[20:16]] <= MEMWBValue_04;
   end
   else if (MEMWBop_04==ALUop_04) begin
     if (MEMWBrd_04!=0) Regs_04[MEMWBrd_04] <= MEMWBValue_04;
   end
   else if ((MEMWBop_04==ADDI_04)||(MEMWBop_04==SUBI_04)) begin
     if (MEMWBIR_04[20:16]!=0) Regs_04[MEMWBIR_04[20:16]] <= MEMWBValue_04;
   end
  end

endmodule
```

## Appendix B: 5-Stage Pipelined MIPS with Forwarding Design Code

```
`timescale 1ns/1ps
module pipe_MIPS32 (clock);
 input clock;

 parameter LW_04    = 6'b100011, // 35
           SW_04    = 6'b101011, // 43
           BEQ_04   = 6'b000100, //  4
           ADDI_04  = 6'b001000, // 8
           SUBI_04  = 6'b001001, // 9
           J_04     = 6'b000010, //  2
           ALUop_04 = 6'b000000, // R-type
           no_op_04 = 32'b000000_00000_00000_00000_00000_000000;
 parameter
           MUL_04   = 6'b011000, // 24
           ADD_04   = 6'b100000, // 32
           SUB_04   = 6'b100010, // 34
           JR_04    = 6'b001000; // 8

 reg [31:0] PC_04;
 reg [31:0] Regs_04[0:31], IMemory_04[0:1023], DMemory_04[0:1023];
 reg [31:0] IFIDIR_04;
 reg [31:0] IDEXA_04, IDEXB_04, IDEXIR_04;
 reg [31:0] EXMEMIR_04, EXMEMALUOut_04, EXMEMB_04;
 reg [31:0] MEMWBIR_04, MEMWBValue_04;

 wire [4:0] IDEXrs_04, IDEXrt_04, EXMEMrd_04, MEMWBrd_04;
 wire [5:0] IDEXop_04, EXMEMop_04, MEMWBop_04;

 wire stall_04, takebranch_04, takejump_04, takejr_04;
 wire bypassAfromMEM_04, bypassBfromMEM_04;
 wire bypassAfromALUinWB_04, bypassBfromALUinWB_04;
 wire [31:0] Ain_04, Bin_04;

 assign IDEXrs_04 = IDEXIR_04[25:21];
 assign IDEXrt_04 = IDEXIR_04[20:16];
 assign IDEXop_04 = IDEXIR_04[31:26];

 assign EXMEMrd_04 = EXMEMIR_04[15:11];
 assign EXMEMop_04 = EXMEMIR_04[31:26];

 assign MEMWBrd_04 = MEMWBIR_04[15:11];
 assign MEMWBop_04 = MEMWBIR_04[31:26];

 assign stall_04 = ( (IDEXop_04==LW_04) &&
          ((IDEXIR_04[20:16]==IFIDIR_04[25:21]) || (IDEXIR_04[20:16]==IFIDIR_04[20:16])) ) ;

 assign takebranch_04 = ((IFIDIR_04[31:26]==BEQ_04) &&
            (Regs_04[IFIDIR_04[25:21]] == Regs_04[IFIDIR_04[20:16]]));

 assign takejump_04 = (IFIDIR_04[31:26]==J_04);
 assign takejr_04   = ((IFIDIR_04[5:0]==JR_04)&&(IFIDIR_04[31:26]==ALUop_04));

 assign bypassAfromMEM_04 = (IDEXrs_04 == EXMEMrd_04) & (IDEXrs_04 != 0) & (EXMEMop_04==ALUop_04);
 assign bypassBfromMEM_04 = (IDEXrt_04 == EXMEMrd_04) & (IDEXrt_04 != 0) & (EXMEMop_04==ALUop_04);
 assign bypassAfromALUinWB_04 = (IDEXrs_04 == MEMWBrd_04) & (IDEXrs_04 != 0) & (MEMWBop_04==ALUop_04);
```

```
  assign bypassBfromALUinWB_04 = (IDEXrt_04 == MEMWBrd_04) & (IDEXrt_04 != 0) & (MEMWBop_04==ALUop_04);
  assign bypassAfromLWinWB_04 = (IDEXrs_04 == MEMWBIR_04[20:16]) & (IDEXrs_04 != 0) &
(MEMWBop_04==LW_04);
  assign bypassBfromLWinWB_04 = (IDEXrt_04 == MEMWBIR_04[20:16]) & (IDEXrt_04 != 0) &
(MEMWBop_04==LW_04);

  assign Ain_04 = bypassAfromMEM_04 ? EXMEMALUOut_04 :      // ALU result from MEM
             (bypassAfromALUinWB_04 | bypassAfromLWinWB_04) ? MEMWBValue_04 :IDEXA_04;

  assign Bin_04 = bypassBfromMEM_04 ? EXMEMALUOut_04 :
             (bypassBfromALUinWB_04 | bypassBfromLWinWB_04) ? MEMWBValue_04 :IDEXB_04;

  always @(posedge clock) begin
    if (~stall_04) begin
      if (takejump_04) begin
        IFIDIR_04 <= no_op_04;
        PC_04     <= ({6'b000000, IFIDIR_04[25:0]}<<2);
      end
      else if (takejr_04) begin
        IFIDIR_04 <= no_op_04;
        PC_04     <= Regs_04[31];
      end
      else if (takebranch_04) begin
        IFIDIR_04 <= no_op_04;
        PC_04     <= PC_04 + 4 + ({{16{IFIDIR_04[15]}},IFIDIR_04[15:0]}<<2);
      end
      else begin
        IFIDIR_04 <= IMemory_04[PC_04>>2];
        PC_04     <= PC_04 + 4;
      end

      IDEXA_04 <= Regs_04[IFIDIR_04[25:21]];
      IDEXB_04 <= Regs_04[IFIDIR_04[20:16]];
      IDEXIR_04<= IFIDIR_04;

      if (IDEXop_04==LW_04 || IDEXop_04==SW_04) begin
        EXMEMALUOut_04 <= IDEXA_04 + {{16{IDEXIR_04[15]}}, IDEXIR_04[15:0]};
      end
      else if (IDEXop_04==ADDI_04) begin
        EXMEMALUOut_04 <= IDEXA_04 + {{16{IDEXIR_04[15]}}, IDEXIR_04[15:0]};
      end
      else if (IDEXop_04==SUBI_04) begin
        EXMEMALUOut_04 <= IDEXA_04 - {{16{IDEXIR_04[15]}}, IDEXIR_04[15:0]};
      end
      else if (IDEXop_04==ALUop_04) begin
        case(IDEXIR_04[5:0])
          ADD_04:
            EXMEMALUOut_04 <= Ain_04 + Bin_04;
          SUB_04:
            EXMEMALUOut_04 <= Ain_04 - Bin_04;
          MUL_04:
            EXMEMALUOut_04 <= Regs_04[IDEXIR_04[25:21]] * Bin_04;
          default: EXMEMALUOut_04 <= 32'hx;
        endcase
      end

      EXMEMIR_04 <= IDEXIR_04;
```

```
    EXMEMB_04  <= IDEXB_04;

   end
   else begin
    if (IDEXop_04==LW_04 || IDEXop_04==SW_04) begin
     EXMEMALUOut_04 <= IDEXA_04 + {{16{IDEXIR_04[15]}}, IDEXIR_04[15:0]};
     EXMEMIR_04 <= IDEXIR_04;
     IDEXIR_04 <= no_op_04;
    end
    else begin
     EXMEMIR_04 <= IDEXIR_04;
     IDEXIR_04 <= no_op_04;
    end
   end


   if (EXMEMop_04==LW_04) begin
    MEMWBValue_04 <= DMemory_04[EXMEMALUOut_04>>2];
   end else if (EXMEMop_04==SW_04) begin
    DMemory_04[EXMEMALUOut_04>>2] <= EXMEMB_04;
   end else if ((EXMEMop_04==ALUop_04)||(EXMEMop_04==ADDI_04)||(EXMEMop_04==SUBI_04)) begin
    MEMWBValue_04 <= EXMEMALUOut_04;
   end
   MEMWBIR_04 <= EXMEMIR_04;

   if (MEMWBop_04==LW_04) begin
    if (MEMWBIR_04[20:16]!=0) Regs_04[MEMWBIR_04[20:16]] <= MEMWBValue_04;
   end
   else if (MEMWBop_04==ALUop_04) begin
    if (MEMWBrd_04!=0) Regs_04[MEMWBrd_04] <= MEMWBValue_04;
   end
   else if ((MEMWBop_04==ADDI_04)||(MEMWBop_04==SUBI_04)) begin
    if (MEMWBIR_04[20:16]!=0) Regs_04[MEMWBIR_04[20:16]] <= MEMWBValue_04;
   end
  end

endmodule
```

## Appendix C: MIPS without forwarding Testbench Code

```
`timescale 1ns / 1ps
module test_mips32_noforwarding;
  reg clk;
  wire [31:0] R1_04,R2_04,R3_04,R4_04,R5_04,R7_04;
  pipe_MIPS32_noforward uut (.clock(clk));
  assign R1_04 = uut.Regs_04[1];
  assign R2_04 = uut.Regs_04[2];
  assign R3_04 = uut.Regs_04[3];
  assign R4_04 = uut.Regs_04[4];
  assign R5_04 = uut.Regs_04[5];
  assign R7_04 = uut.Regs_04[7];
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end
  integer k_04;
  initial begin
    uut.PC_04 = 0;
    uut.IFIDIR_04=32'h00000000;
    uut.IDEXIR_04=32'h00000000;
    uut.EXMEMIR_04=32'h00000000;
    uut.MEMWBIR_04=32'h00000000;
    uut.Regs_04[0] = 32'd0;
    for (k_04 = 1; k_04 < 32; k_04 = k_04 + 1)
      uut.Regs_04[k_04] = 32'd0;
    #1
    // 0: add $1,$0,$0 => opcode=0, rs=0, rt=0, rd=1, funct=32 => 0x00208020
    uut.IMemory_04[0] = 32'h00208020;
    // Index 1 (Addr 4): beq $7,$0, 14
    uut.IMemory_04[1] = 32'h10E0000E; // <-- Offset is now 14 (0xE)

    // Index 2 (Addr 8): lw $2, 0($3)
    uut.IMemory_04[2] = 32'h8C620000;

    // Index 3 (Addr 12): NOP
    uut.IMemory_04[3] = 32'b0;

    // Index 4 (Addr 20): lw $4, 0($5)
    uut.IMemory_04[4] = 32'h8CA40000;

    // Index 5 (Addr 16): NOP
    uut.IMemory_04[5] = 32'b0;

    // Index 6 (Addr 24): NOP
    uut.IMemory_04[6] = 32'b0;

    // Index 7 (Addr 28): NOP
    uut.IMemory_04[7] = 32'b0;

    // Index 8 (Addr 32): mul $2,$2,$4
    uut.IMemory_04[8] = 32'h00441018;

    // Index 9 (Addr 36): NOP
    uut.IMemory_04[9] = 32'b0;
```

```
    // Index 10 (Addr 40): NOP
    uut.IMemory_04[10] = 32'b0;

    // Index 11 (Addr 44): NOP
    uut.IMemory_04[11] = 32'b0;

    // Index 12 (Addr 48): add $1,$1,$2
    uut.IMemory_04[12] = 32'h00220820;

    // Index 13 (Addr 52): subi $7,$7,#1
    uut.IMemory_04[13] = 32'h24E70001;

    // Index 14 (Addr 56): addi $3,$3,#4
    uut.IMemory_04[14] = 32'h20630004;

    // Index 15 (Addr 60): addi $5,$5,#4
    uut.IMemory_04[15] = 32'h20A50004;

    // Index 16 (Addr 64): j 1
    uut.IMemory_04[16] = 32'h08000001;

    // Index 17 (Addr 68): jr $31
    uut.IMemory_04[17] = 32'h03E00008;
    //   r3=100, r5=200, r7=9, r31=0
    uut.Regs_04[3]  = 32'd100;
    uut.Regs_04[5]  = 32'd200;
    uut.Regs_04[7]  = 32'd9;
    uut.Regs_04[31] = 32'd0;

    uut.DMemory_04[25] = 32'd0;   // M[100]
    uut.DMemory_04[26] = 32'd1;   // M[104]
    uut.DMemory_04[27] = 32'd7;   // M[108]
    uut.DMemory_04[28] = 32'd5;   // M[112]
    uut.DMemory_04[29] = 32'd1;   // M[116]
    uut.DMemory_04[30] = 32'd5;   // M[120]
    uut.DMemory_04[31] = 32'd0;   // M[124]
    uut.DMemory_04[32] = 32'd0;   // M[128]
    uut.DMemory_04[33] = 32'd4;   // M[132]

    uut.DMemory_04[50] = 32'd4;   // M[200]
    uut.DMemory_04[51] = 32'd5;   // M[204]
    uut.DMemory_04[52] = 32'd1;   // M[208]
    uut.DMemory_04[53] = 32'd9;   // M[212]
    uut.DMemory_04[54] = 32'd5;   // M[216]
    uut.DMemory_04[55] = 32'd9;   // M[220]
    uut.DMemory_04[56] = 32'd4;   // M[224]
    uut.DMemory_04[57] = 32'd4;   // M[228]
    uut.DMemory_04[58] = 32'd8;   // M[232]

    #1999;
    $finish;
  end
    initial begin
    $dumpfile("test_mips32_noforwarding.vcd");
    $dumpvars(0, test_mips32_noforwarding);
  end
endmodule
```

## Appendix D: MIPS with forwarding Testbench Code

```
`timescale 1ns / 1ps
module test_MIPS32;
  reg clk;
  wire [31:0] R1_04,R2_04,R3_04,R4_04,R5_04,R7_04;
  pipe_MIPS32 uut (.clock(clk));
  assign R1_04 = uut.Regs_04[1];
  assign R2_04 = uut.Regs_04[2];
  assign R3_04 = uut.Regs_04[3];
  assign R4_04 = uut.Regs_04[4];
  assign R5_04 = uut.Regs_04[5];
  assign R7_04 = uut.Regs_04[7];
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end
  integer k_04;
  initial begin
    uut.PC_04 = 0;
    uut.IFIDIR_04=32'h00000000;
    uut.IDEXIR_04=32'h00000000;
    uut.EXMEMIR_04=32'h00000000;
    uut.MEMWBIR_04=32'h00000000;
    uut.Regs_04[0] = 32'd0;
    for (k_04 = 1; k_04 < 32; k_04 = k_04 + 1)
      uut.Regs_04[k_04] = 32'd0;
    #1
    // 0: add $1,$0,$0 => opcode=0, rs=0, rt=0, rd=1, funct=32 => 0x00208020
    uut.IMemory_04[0] = 32'h00208020;

    // 1: beq $7,$0, offset=8 => opcode=4, rs=7, rt=0, imm=7 => 0x10E00007
    uut.IMemory_04[1] = 32'h10E00007;

    // 2: lw $2, 0($3) => opcode=35=100011, rs=3, rt=2, imm=0 => 0x8C620000
    uut.IMemory_04[2] = 32'h8C620000;

    // 3: lw $4, 0($5) => 0x8CA40000
    uut.IMemory_04[3] = 32'h8CA40000;

    // 4: mul $2,$2,$4 => R-type: op=0, rs=2, rt=4, rd=2, funct=24 => 0x00441018
    uut.IMemory_04[4] = 32'h00441018;

    // 5: add $1,$1,$2 => R-type: op=0, rs=1, rt=2, rd=1, funct=32 => 0x00220820
    uut.IMemory_04[5] = 32'h00220820;

    // 6: subi $7,$7,#1 => opcode=9=001001, rs=7, rt=7, imm=1 => 0x24E70001
    uut.IMemory_04[6] = 32'h24E70001;

    // 7: addi $3,$3,#4 => opcode=8=001000, rs=3, rt=3, imm=4 => 0x20630004
    uut.IMemory_04[7] = 32'h20630004;

    // 8: addi $5,$5,#4 => 0x20A50004
    uut.IMemory_04[8] = 32'h20A50004;

    // 9: j loop => opcode=2=000010, target=1 => => 0x08000001
    uut.IMemory_04[9] = 32'h08000001;
```

```
    // 10: jr $31 => R-type: op=0, rs=31, rt=0, rd=0, funct=8 => 0x03E00008
    uut.IMemory_04[10] = 32'h03E00008;

    //   r3=100, r5=200, r7=9, r31=0
    uut.Regs_04[3]  = 32'd100;
    uut.Regs_04[5]  = 32'd200;
    uut.Regs_04[7]  = 32'd9;
    uut.Regs_04[31] = 32'd0;

    // vector A
    uut.DMemory_04[25] = 32'd0;   // M[100]
    uut.DMemory_04[26] = 32'd1;   // M[104]
    uut.DMemory_04[27] = 32'd7;   // M[108]
    uut.DMemory_04[28] = 32'd5;   // M[112]
    uut.DMemory_04[29] = 32'd1;   // M[116]
    uut.DMemory_04[30] = 32'd5;   // M[120]
    uut.DMemory_04[31] = 32'd0;   // M[124]
    uut.DMemory_04[32] = 32'd0;   // M[128]
    uut.DMemory_04[33] = 32'd4;   // M[132]

    // vector B
    uut.DMemory_04[50] = 32'd4;   // M[200]
    uut.DMemory_04[51] = 32'd5;   // M[204]
    uut.DMemory_04[52] = 32'd1;   // M[208]
    uut.DMemory_04[53] = 32'd9;   // M[212]
    uut.DMemory_04[54] = 32'd5;   // M[216]
    uut.DMemory_04[55] = 32'd9;   // M[220]
    uut.DMemory_04[56] = 32'd4;   // M[224]
    uut.DMemory_04[57] = 32'd4;   // M[228]
    uut.DMemory_04[58] = 32'd8;   // M[232]

    #1999;
    $finish;
  end

  initial begin
    $dumpfile("test_MIPS32.vcd");
    $dumpvars(0, test_MIPS32);
  end
endmodule
```